A Bounded Protocol Audit of Parsl with TLA+:
Ownership, Retry, and Message Correlation in Distributed Workflows

TLA+ Parsl Audit Project

October 2026

Abstract

Workflow APIs make a distributed execution graph look like a local Python program, but correctness depends on ownership transfers between futures, executors, workers, providers, transports, and staging services. This paper presents a small executable TLA+ abstraction of Parsl designed for a bounded protocol audit rather than a complete implementation model. The abstraction separates a logical task from its physical execution attempts, transport messages, result generations, and monitoring state. We use a source–model–runtime evidence chain: a source transition is encoded in a *Current* model, explored with TLC, exercised where possible by a focused Parsl runtime probe, and compared with a candidate *Fixed* protocol. The communication-first audit covers dependency blocking, futures, retry, worker and provider failure, timeout and stale results, provisioning, scaling, memoization, and data readiness. The repository contains 676 TLA+ modules, 1,478 TLC configurations, 486 foundational runtime entries, and a 318-record risk ledger. The central result is a reusable safety vocabulary: failed communication must preserve ownership or make failure explicit; results must carry attempt identity; and terminal state must be monotone. The ledger records source-backed protocol risks, not claims that every entry is a confirmed upstream defect. The paper reports three detailed counterexample patterns and gives commands for reproducing the bounded verification and runtime checks.

Keywords: Parsl, TLA+, TLC, workflow systems, futures, asynchronous messaging, retry, formal methods

1 Introduction

Parsl is a Python workflow system in which an application builds a dataflow graph and a dataflow kernel (DFK) arranges execution on local or remote resources. The user sees apps and futures; the implementation crosses several asynchronous boundaries. A task can be accepted by the DFK, queued by an executor, serialized into a message, launched by a worker, reported to a result collector, and finally resolved as a future. Providers and staging components introduce additional boundaries for resources and files. Each boundary can fail independently of the user function. This architecture places Parsl in the broader family of dataflow and scientific workflow systems such as Pegasus, Swift, Dask, and Ray [1, 3, 4, 5, 6].

This creates a characteristic verification problem. A call stack is a poor description of the system's safety properties: a task may be removed from one queue before it is accepted by the next component, and a delayed result may arrive after a retry has become current. The useful question is instead *who owns the task now, which attempt generated this message, and has terminal state already been published?* We therefore model the following identity chain:

logical task – physical attempt – transport message – Future and monitoring record

Figure 1: Ownership and identity flow used by the abstraction.

The motivating example is a task-dispatch send failure in the HTEX path. A source-like sequence dequeues a pending task and then calls a socket send. If the send raises, the task is no longer in the pending queue, is not owned by a manager, and has not been terminalized. A

Future can remain pending indefinitely. This is a small state machine, but it is difficult to see in a conventional unit test because it requires a particular interleaving at a process boundary.

The audit asks three questions:

1. How can ownership be preserved when a communication operation fails between two Parsl components?
2. How should retry results be correlated with the current physical attempt so that late results cannot overwrite a newer outcome?
3. Which asynchronous races change an observable terminal state, and which can be safely isolated as stale, malformed, or duplicate input?

The contributions are fourfold. First, we give a compact executable abstraction that keeps logical tasks separate from attempts, messages, and futures. Second, we define a protocol-oriented set of safety properties covering dependencies, ownership, retries, terminality, and stale messages. Third, we connect bounded TLC exploration to deterministic Python probes and a source-indexed ledger. Finally, we show representative communication and asynchronous counterexample patterns and state the limits of what they establish.

The rest of the paper is organized as follows. Section 2 describes Parsl's failure surface. Section 3 explains the audit method and evidence discipline. Section 4 presents the TLA+ state machine and invariants. Section 5 maps the abstraction to source paths and runtime probes. Section 6 gives three case studies. Section 7 evaluates the artifact and regression suites. Sections 8 and 9 discuss implications and threats to validity, and Section 10 gives reproduction commands and the conclusion.

2 Parsl Background and Failure Surface

2.1 Logical workflow and physical execution

An app invocation creates a logical task with dependencies on input futures. The DFK does not execute the Python body directly in the caller's thread: it schedules work through an executor, which may use a manager and remote workers. A provider requests and releases resources from a batch scheduler or cloud service. DataFuture and staging machinery may make a file available only after a transfer has completed. Monitoring and persistence record progress independently of the application's future callbacks.

For verification we use five identities:

Logical task the stable user-visible unit and its dependency set.
Physical attempt one execution generation of that task; retries create a new generation.
Message a serialized command, result, acknowledgement, heartbeat, or provider event.
Future the completion object exposed to the caller; it must resolve at most once.
Monitoring record an operational view of state, useful for diagnosis but not a replacement for task ownership.

2.2 Boundaries and failure modes

Table 1 summarizes the boundaries selected for the communication-first scope. The source names are intentionally broad: the TLA+ model abstracts call-level behavior instead of copying implementation internals.

The important distinction is between a failed operation and an explicit terminal failure. A send exception is not itself a task result. If the implementation removes the only owner and then lets the exception escape, the logical task is lost. Conversely, a delayed result after a timeout is not necessarily a new result for the logical task: it may belong to an obsolete attempt.

Table 1: Source boundaries represented by the model.

Boundary	Observable transition	Failure represented
DFK/executor	ready task becomes queued or dispatched	dequeue before failed send; duplicate dispatch
executor/worker	command becomes an execution attempt	worker loss, timeout, retry, stale completion
serialization/ZMQ	object becomes multipart message and back	malformed frame, send failure, late or duplicate message
provider/scheduler	resource request becomes running allocation	provisioning failure, cancellation, scale-up/down race
staging/file	transfer becomes readable data	partial file, failed transfer, late publication
monitoring/ heartbeat	event becomes observation or liveness evidence	ACK loss, stale heartbeat, bookkeeping

3 Audit Methodology

3.1 Source-to-model-to-runtime workflow

The audit uses a four-stage discipline. We first identify a source transition and its ownership boundary. We then encode a minimal source-like transition in a *Current* TLA+ module. The Current specification is expected to violate a named invariant for the selected failure. A *Fixed* module adds the smallest protocol change that should preserve the invariant: retain a queue owner until acknowledgement, tag results with an attempt generation, isolate malformed input, or snapshot a collection before callback fan-out. Finally, a Python probe exercises the real Parsl path or a deterministic test double when the real boundary is difficult to control. This separation between model checking and proof-oriented validation follows established industrial TLA+ practice [16, 14, 15].

source transition – Current TLC trace – runtime probe – Fixed invariant – ledger record

Figure 2: Evidence ladder connecting source review, model checking, and runtime observation.

This evidence chain prevents a model-only observation from being presented as a production defect. The ledger stores the source location, model and configuration, expected result, probe type, candidate protocol, and limitations. A record is a source-backed protocol risk unless the project has independently confirmed it in an upstream release.

3.2 Finite scope and stopping rule

The project deliberately does not simulate all Python objects or every backend detail in the Parsl paper. Constants bound tasks, attempts, workers, messages, and queue lengths. The selected scope stops when each chosen communication path has (i) a source note, (ii) a Current and Fixed model or an explicit reason why one is not meaningful, (iii) a runtime probe where feasible, and (iv) a reproducible ledger entry. Expanding to another provider is a scope change, not an implicit promise of exhaustive coverage.

TLC configuration suites are split into foundational Fixed cases and focused Current cases. The foundational suite is a regression gate: all cases should pass. Current cases are negative tests: they should produce the documented counterexample. Simulation bounds explore interleavings and are not an unbounded proof [12, 13]. Runtime probes complement rather than replace TLC: probes exercise Python and socket behavior, while TLC explores combinations that are hard to schedule reliably.

4 TLA+ Abstraction

4.1 State variables

The core model uses finite sets of tasks, attempts, and message identifiers. The key state variables are shown below; extended modules refine them for providers, staging, monitoring, and cancellation.

Table 2: Core state variables and their interpretation.

Variable	Meaning
`taskState[t]`	logical task state: blocked, ready, queued, running, succeeded, failed, or cancelled
`attempt[t]`	current physical-attempt generation for logical task t
`owner[t]`	component that currently owns delivery or execution responsibility
`msgState[m]`	created, sent, delivered, rejected, duplicate, stale, or dropped
`msgAttempt[m]`	attempt generation carried by result/command message m
`futureState[t]`	unresolved, resolved-success, resolved-failure, or cancelled
`retryCount[t]`	number of retries consumed by logical task t
`monitorState[t]`	last operational observation; it cannot reopen a terminal Future

Dependencies are represented by a finite relation `deps` between finite task sets. A task may enter `ready` only when all dependencies have succeeded and their result values are available. Memoization adds a cache keyed by an input identity; a cache hit produces a result without creating a new physical attempt, but still resolves the logical Future exactly once.

4.2 Actions

The following pseudocode is a compressed version of the executable modules. It omits set-builder syntax that is not needed to understand the protocol boundary.

Listing 1: Core Current/Fixed protocol actions.

```
Dispatch(t) ==
  taskState[t] = "queued" /\
  LET m == NewCommand(t, attempt[t]) IN
    IF SendSucceeds(m)
    THEN owner[t] := "manager" /\ msgState[m] := "sent"
    ELSE Current: owner[t] := "none" /\ msgState[m] := "dropped"
         Fixed: owner[t] := "executor" /\ taskState[t] := "queued"

Timeout(t) ==
  taskState[t] = "running" /\ retryCount[t] < RetryLimit /\
  attempt[t] := attempt[t] + 1 /\ taskState[t] := "queued"

DeliverResult(m) ==
```

```
IF msgAttempt[m] = attempt[msgTask[m]]
THEN resolve Future and publish terminal monitoring state
ELSE msgState[m] := "stale" /\ no logical state changes
```

The model treats a message as a protocol object, not merely as a value. A multipart message can fail at framing or decoding before a task is touched. A duplicate message is consumed idempotently; a stale message is rejected using the current attempt generation. The same rule applies to worker results, manager-loss synthetic results, and late staging notifications. These assumptions are consistent with the atomic multipart and peer-ordering guarantees specified by ZMTP, but the application-level retry and ownership policy remains outside the transport [25, 24, 20].

4.3 Safety properties

Table 3 lists the invariants used in the foundational suites. They are safety properties: a finite counterexample is sufficient to disprove one, while a passing bounded run only supports the stated finite instance.

Table 3: Main invariants and the failure they exclude.

Invariant	Intended guarantee
DependencySafety	a task cannot execute or publish success before all dependencies succeed
OwnershipSafety	every non-terminal task is queued, assigned, or explicitly owned by a retry/failure path
RetryBound	retries never exceed the configured limit and each retry advances the generation
ResultConsistency	only a result from the current attempt can resolve the logical Future
TerminalStability	success, failure, and cancellation are monotone and cannot be reopened
StaleIsolation	stale, duplicate, and malformed messages do not mutate current logical state
MemoizationSafety	a cache hit is equivalent to one successful result, not a second completion
ReadinessSafety	a task consuming staged data observes only atomically published data

These properties intentionally cross component boundaries. For example, ResultConsistency is not an executor-only property: it constrains the relation between worker output, serialization metadata, the result collector, and the Future callback.

5 Source and Runtime Evidence

5.1 Mapping to Parsl components

The repository maps DFK/Future paths to logical state and terminality; HTEX interchange, manager, worker, and message-manager paths to frames, ownership, and ACKs; provider adapters to request, running, cancellation, and scale; and DataManager/file-transfer paths to publication and readiness. The model follows stable interfaces (enqueue, send, launch, retry, publish, observe) rather than copying every backend call frame. The staging and provenance vocabulary is compatible with the workflow provenance literature and W3C PROV data model [26, 27, 28].

5.2 Probe design

Probes are small Python programs under the runtime probe tree. They use deterministic doubles for socket send failures, malformed frames, provider responses, callback mutation, and file publication. Where practical, a probe enters a real in-process Parsl path and replaces only the failure boundary. This is particularly useful for confirming that a source-level exception is reachable without requiring a cluster. Probe results are classified as observed, expected, or blocked; a blocked probe is never silently converted into a model claim.

One representative probe constructs a fake transport whose first send raises and whose second send succeeds. Similar doubles inject a delayed result, malformed command, or ACK failure. Each probe is small enough to map to one action and invariant.

To make this claim auditable, the repository includes a machine-readable source-to-model manifest (`docs/source-model-evidence.json`) and a validator (`scripts/verify_source_coverage.py`). The manifest records nine representative boundaries, their inspected Parsl source paths, model files, runtime probes, and safety properties. The validator checks that every declared model and probe is present and reports the current inventory (676 TLA+ modules and 490 runtime probe files). This is traceability evidence, not line coverage: it demonstrates that the central protocol paths have linked artifacts while leaving backend-specific depth and external source revision explicit.

6 Findings and Case Studies

6.1 Case 1: HTEX dispatch send failure (BUG-335)

Source transition. A pending task is selected and removed from the queue before the interchange sends a command to a manager. The send operation can raise due to a closed or broken ZeroMQ socket. The exception path does not itself create a task result.

Current trace. Table 4 gives the minimal four-state trace. The failure is not a worker failure: the task never reached a worker, so retry logic attached to execution does not necessarily run.

Table 4: Representative Current counterexample for BUG-335.

State	Queue	Owner	Observation
0	task t pending	executor	Future unresolved
1	empty	executor	dequeue completed
2	empty	none	socket send raises
3	empty	none	no result, retry, or terminal failure

Violated property. OwnershipSafety fails because a non-terminal task has no owner. DependencySafety and RetryBound are irrelevant until ownership is recovered; this explains why a test that only checks eventual worker retry can miss the case.

Runtime evidence and candidate fix. The deterministic send-failure probe reaches the same transition shape. The candidate protocol retains executor ownership until the send is acknowledged, or converts the transport exception into an explicit terminal transport failure. A bounded Fixed model passes OwnershipSafety and TerminalStability for the selected constants. The ledger records this as a source-backed risk, not as evidence that every Parsl deployment loses tasks.

dequeue – send raises – owner: none

required repair: retain executor ownership or publish explicit terminal transport failure

Figure 3: The minimal ownership-loss shape in the BUG-335 counterexample.

6.2 Case 2: retry and stale result correlation

Scenario. Attempt 0 starts, times out, and causes attempt 1 to become current. Attempt 1 succeeds. A delayed result from attempt 0 then arrives through the result collector. If the collector keys only on the logical task identifier, the late result can overwrite the successful value or invoke a second completion callback.

attempt 0: running – timeout – attempt 1: running – success;

late attempt-0 result – reject as stale, consume idempotently, no state change

Figure 4: Retry timeline and stale-result rejection.

Model result. The model carries the attempt generation in every command and result message. DeliverResult compares the message generation with the current generation. Equality resolves the Future once; a smaller generation changes only the message state to `stale`. The Fixed model therefore satisfies ResultConsistency, TerminalStability, and StaleIsolation. The scenario also exposes why a retry counter alone is insufficient: the counter bounds the number of attempts but does not identify which result is current. This is the same distinction made by logical clocks and consistent distributed observations [17, 18, 19].

Implementation implication. Correlation metadata must survive serialization and route changes. It is not enough to add a generation field to an executor-local object if the field is dropped when a result is framed, decoded, or converted into a synthetic manager-loss result.

6.3 Case 3: callback mutation during failure fan-out

Scenario. A failure path iterates a live task dictionary and synchronously invokes Future callbacks. A callback removes an entry, adds a replacement, or triggers cancellation while the iteration is in progress. The container mutation can raise an exception and leave unrelated Futures unresolved.

Model and fix. The model represents the registry as a finite set and the callback as an action that can mutate it. The Current action permits iteration and mutation in one transition and can leave a pending Future without a terminal event. The candidate Fixed protocol snapshots the entries before callback invocation, then records each terminal transition independently. This is an asynchronous terminality risk rather than a transport framing risk, but it uses the same principle: the operation that loses control must retain explicit ownership until terminal state is published.

The same vocabulary also produced focused refinements for malformed command ingress (BUG-336), command-reply send failure (BUG-337), heartbeat ACK send failure (BUG-338), and a manager-loss synthetic result that fails during transport (BUG-004 refinement). In each case, the safe alternatives are to preserve the old owner, install an explicit retry owner, or publish a terminal failure.

Table 5: Case-study evidence chains.

Case	Current transition	Invariant/counterexample	Fixed protocol
BUG-335	dequeue then failed command send	OwnershipSafety; task has no owner	retain ownership until send acknowledgement
Stale result	attempt 0 result after attempt 1 success	ResultConsistency; Future reopens or changes value	carry and check attempt generation
Callback mutation	live registry changes during fan-out	TerminalStability; unrelated Future remains pending	snapshot entries and isolate callbacks

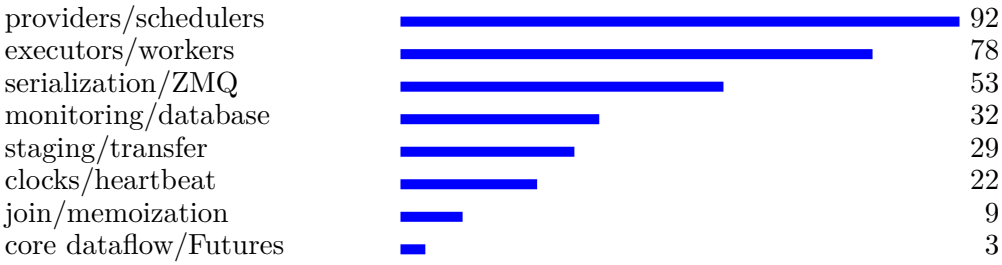


Figure 5: Ledger records by broad category; counts indicate audit allocation, not defect prevalence.

7 Evaluation

7.1 Artifact inventory

The current repository inventory is shown in Table 6. Counts are generated from the checked-in trees and are included to make scope changes visible. A ledger record is not necessarily a unique defect: one numeric identifier can have a refinement record, and a single source pattern can have multiple model/probe artifacts.

Table 6: Repository and regression inventory.

Measure	Value
TLA+ modules	676
TLC configuration files	1,478
Foundational TLC configurations	754
Runtime probe entries	486
Runtime probe files	490
Recorded unittest methods	761
Bug ledger records	318
Approximate unique numeric BUG identifiers	317

7.2 Regression results

The latest complete foundational regression passed 754/754 TLC configurations and 486/486 runtime entries. The normal gate runs Fixed/safe configurations. Focused Current configurations are negative tests and are expected to emit their documented counterexamples; they are not counted as failures of the safe gate. The runtime suite uses deterministic doubles and selected in-process Parsl paths, so its result is reproducible without a production cluster.

The model families are summarized below. The table describes behavioral scope, not a claim that each backend has equal source depth.

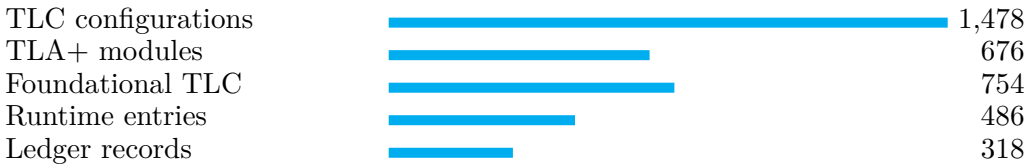


Figure 6: Relative scale of the main audit artifacts. Bars are linearly scaled to the TLC configuration count.

Table 7: Model families in the communication-first audit.

Family	Behaviors	Representative safety focus
Dataflow	dependencies, futures, join, memoization	dependency and terminality safety
Execution	workers, attempts, retry, cancellation, timeout	retry bound and result consistency
Transport	serialization, multipart frames, routes, ACKs	ownership and stale isolation
Provisioning	provider request, status, scale, cleanup	explicit resource ownership
Staging	transfer, partial file, atomic publication	readiness and stale transfer isolation
Operations	monitoring, heartbeat, persistence, shutdown	observation cannot reopen terminal state

8 Discussion

8.1 Ownership is a cross-component abstraction

The recurring pattern is an ownership gap. A queue, socket, worker, provider, or staging service may be treated as if it has accepted responsibility before it has acknowledged the transfer. The implementation then has no place to retry from when the next operation fails. Modeling ownership explicitly makes the gap visible without reproducing every Python call frame. This complements lineage-based recovery in MapReduce and Spark, and ownership-based recovery in distributed futures systems [7, 8, 9].

8.2 Terminality is stronger than eventual observation

Monitoring events, heartbeats, and database rows are observations. They can arrive late, be duplicated, or disappear during shutdown. A Future's terminal state must therefore be monotone and independent of whether a later monitoring event is processed. This separation also clarifies why a successful retry must not be reopened by an old result or a stale failure notification.

Candidate Fixed protocols are designs, not patches. They identify where an implementation should retain ownership, carry correlation metadata, or isolate callbacks. A maintainer would still need to choose API compatibility, logging, shutdown, and performance behavior before applying a fix.

9 Threats to Validity and Limitations

The most important limitation is scope. This is not a full simulation of every component described in the Parsl paper. It is a small executable model of the most error-prone interactions, with communication and asynchronous terminality prioritized. The scope is explicit so that additional components can be added as separate, reviewable evidence rather than silently inflating the claim. The paper also avoids implying that coordination is free or always solvable under arbitrary failures; classic results establish fundamental limits for consensus and consistency under faults [21, 22].

10 Reproducibility and Conclusion

The repository contains the TLA+ modules, TLC configurations, runtime probes, source inventory, coverage matrix, bug ledger, Markdown report, and this compiled PDF. From a clean checkout, the foundational commands are:

```
PYTHONPATH=/tmp/parsl-source \\
  PYTHON_BIN=/tmp/parsl-venv/bin/python \\
  PARSL_SOURCE=/tmp/parsl-source \\
  bash scripts/runtime_foundational_smoke.sh

JAVA_BIN=/path/to/java \\
  TLA_JAR=/path/to/tla2tools.jar \\
  TLC_SIMULATE=100 \\
  bash scripts/tlc_foundational_smoke.sh
```

The exact paths can be changed for a local installation; the scripts report missing tools rather than silently skipping a suite. Focused Current runners are useful when reviewing a case study, while the foundational scripts are the release-style regression gate. The interactive companion under `site/dist` provides a presentation view, but the paper does not depend on JavaScript.

This bounded audit shows why a small model can be useful even for a large workflow system. Parsl's surface area is broad, but the safety vocabulary is compact: preserve ownership across failed communication, correlate every result with its physical attempt, isolate malformed or stale input, and make terminal state monotone. TLC supplies interleaving counterexamples; runtime probes connect those traces to executable boundaries; and the ledger preserves the distinction between a plausible risk and a confirmed defect. These principles are the practical outcome of the project and the starting point for deeper audits of specific backends.

A Boundary Catalog and Review Trail

This appendix makes the abstraction boundary explicit. Each row starts from a concrete Parsl source surface, names the state retained by the model, and lists the evidence that a reviewer can open. The source paths refer to the Parsl checkout inspected for the audit; the model and probe paths are in this repository. The catalog is intentionally a protocol map rather than a claim of Python line coverage.

A.1 How to audit one row

For each row, a reviewer can follow the same five-step trail:

1. open the source path and identify the ownership-changing operation;
2. open the named TLA+ module and locate the corresponding action and Current/Fixed split;
3. open the runtime probe and identify the injected failure or adversarial input;
4. run the relevant TLC configuration and inspect the invariant or counterexample trace;
5. compare the result with the ledger entry, including its limitation and confidence label.

The machine-readable version of this catalog is `docs/source-model-evidence.json`; `scripts/verify_source_coverage.py` checks that every declared model and probe exists. A passing check establishes repository traceability. It does not establish that every function in the external Parsl source tree has been modeled.

B Action and Invariant Cross-Reference

The model uses a small vocabulary across otherwise different backends. Table 10 shows how concrete implementation actions are normalized before they are checked.

The normalization is what makes cross-component comparison possible. A provider poll and a ZMQ receive are not implemented the same way, but both can be checked for late input, ownership loss, and terminal-state monotonicity. Conversely, the catalog preserves the places where the abstraction is intentionally weak: queue bounds, symbolic bytes, deterministic scheduler responses, and the absence of arbitrary Python heap mutation.

References

[1] Y. Babuji et al., "Parsl: Pervasive Parallel Programming in Python," *HPDC*, 2019, doi:10.1145/3307681.3325400.

[2] E. Deelman et al., "Pegasus: A Framework for Mapping Complex Scientific Workflows onto Distributed Systems," *Scientific Programming*, 13(3), 219–237, 2005, doi:10.1155/2005/128026.

[3] E. Deelman et al., "Pegasus, a workflow management system for science automation," *Future Generation Computer Systems*, 46, 17–35, 2015, doi:10.1016/j.future.2014.10.008.

[4] M. Wilde et al., "Swift: A language for distributed parallel scripting," *Parallel Computing*, 37(9), 633–652, 2011, doi:10.1016/j.parco.2011.05.005.

[5] M. Rocklin, "Dask: Parallel Computation with Blocked algorithms and Task Scheduling," *Proceedings of the 14th Python in Science Conference*, 126–132, 2015, doi:10.25080/Majora-7b98e3ed-013.

[6] P. Moritz et al., "Ray: A Distributed Framework for Emerging AI Applications," *OSDI*, 561–577, 2018.

[7] J. Dean and S. Ghemawat, "MapReduce: Simplified Data Processing on Large Clusters," *OSDI*, 137–150, 2004.

[8] M. Zaharia et al., "Resilient Distributed Datasets: A Fault-Tolerant Abstraction for In-Memory Cluster Computing," *NSDI*, 15–28, 2012.

[9] S. Wang et al., "Ownership: A Distributed Futures System for Fine-Grained Tasks," *NSDI*, 671–686, 2021.

[10] L. Lamport, "The Temporal Logic of Actions," *ACM TOPLAS*, 16(3), 872–923, 1994, doi:10.1145/177492.177726.

[11] L. Lamport, *Specifying Systems: The TLA+ Language and Tools for Hardware and Software Engineers*. Addison-Wesley, 2002.

[12] Y. Yu, P. Manolios, and L. Lamport, "Model Checking TLA+ Specifications," *CHARME*, LNCS 1703, 54–66, 1999.

[13] I. Konnov, J. Kukovec, and T.-H. Tran, "TLA+ Model Checking Made Symbolic," *Proceedings of the ACM on Programming Languages*, 3(OOPSLA), 123:1–123:30, 2019, doi:10.1145/3360549.

[14] D. Cousineau et al., "TLA+ Proofs," *FM*, LNCS 7436, 147–154, 2012, doi:10.1007/978-3-642-32759-9_13.

[15] K. Chaudhuri, D. Doligez, L. Lamport, and S. Merz, "Verifying Safety Properties with the TLA+ Proof System," *IJCAR*, LNCS 6173, 142–148, 2010.

[16] C. Newcombe et al., "How Amazon Web Services Uses Formal Methods," *Communications of the ACM*, 58(4), 66–73, 2015, doi:10.1145/2699417.

[17] L. Lamport, "Time, Clocks, and the Ordering of Events in a Distributed System," *Communications of the ACM*, 21(7), 558–565, 1978, doi:10.1145/359545.359563.

[18] K. M. Chandy and L. Lamport, "Distributed Snapshots: Determining Global States of Distributed Systems," *ACM TOCS*, 3(1), 63–75, 1985, doi:10.1145/214451.214456.

[19] M. Herlihy and J. M. Wing, "Linearizability: A Correctness Condition for Concurrent Objects," *ACM TOPLAS*, 12(3), 463–492, 1990, doi:10.1145/78969.78972.

[20] K. P. Birman and T. A. Joseph, "Reliable Communication in the Presence of Failures," *ACM TOCS*, 5(1), 47–76, 1987, doi:10.1145/7351.7478.

[21] M. J. Fischer, N. A. Lynch, and M. S. Paterson, "Impossibility of Distributed Consensus with One Faulty Process," *JACM*, 32(2), 374–382, 1985, doi:10.1145/3149.214121.

[22] S. Gilbert and N. Lynch, "Brewer's Conjecture and the Feasibility of Consistent, Available, Partition-Tolerant Web Services," *ACM SIGACT News*, 33(2), 51–59, 2002, doi:10.1145/564585.564601.

[23] D. Ongaro and J. Ousterhout, "In Search of an Understandable Consensus Algorithm," *USENIX ATC*, 305–319, 2014.

[24] P. Hintjens, *ZeroMQ: The Guide*. O'Reilly, 2013.

[25] ZeroMQ Community, "ZMTP 3.1 Specification (RFC 37)," 2013.

[26] L. Moreau and P. Missier, eds., "PROV-DM: The PROV Data Model," W3C Recommendation REC-prov-dm-20130430, 2013.

[27] V. Cuevas-Vicenttín et al., "Scientific Workflows and Provenance: Introduction and Research Opportunities," arXiv:1311.4610, 2013.

[28] J. Chen et al., "A provenance model for control-flow driven scientific workflows," *Data & Knowledge Engineering*, 131–132, 101877, 2021.

Table 8: Threats to validity and mitigations.

Threat	Mitigation or interpretation
Finite state space	Constants bound tasks, attempts, queues, and workers. A passing TLC run is evidence for that instance, not an unbounded proof.
Abstract Python state	The model uses symbolic values rather than arbitrary heaps, descriptors, or operating-system scheduling. Runtime probes cover selected concrete boundaries.
Deterministic failures	Socket and provider doubles make races repeatable but do not estimate production frequency.
Uneven backend depth	Communication paths receive priority; provider adapters and less common executors are represented at a coarser interface level.
Version drift	Source notes identify the revision inspected. Future Parsl changes require rerunning inventory and probes.
Risk versus defect	Ledger records are source-backed protocol risks unless independently confirmed; the paper avoids claiming that all records are upstream bugs.
Liveness	The primary properties are safety properties. A model can show that ownership is not lost without proving eventual completion under a fair network.

Table 9: Detailed source-to-abstraction boundary catalog.

Boundary	Parsl source surface	Model state and transition	Evidence and remaining abstraction
DFK/Futures	`parsl/dataflow/dflow.py`; `futures.py`	logical task, dependency set, Future state; blocked – ready – queued	`ParslEndToEnd.tla` and dependency probe; arbitrary user object graphs are symbolic
HTEX dispatch	interchange module	queue ownership, manager route, command message; dequeue before/after send	task-dispatch model; send-failure probe; socket internals are abstract
ZMQ transport	serialize package; HTEX framing	frame metadata, route, payload identity, attempt tag; decode/reject/stale	serialization models and frame probes; bytes and queues are bounded
Manager/worker	HTEX manager and worker modules	attempt, result, worker liveness; launch/timeout/retry/complete	duplicate-result models and probes; OS scheduling is not modeled
Providers	`parsl/providers/`; status polling	resource owner, submit response, scheduler state; request, running, cancel, scale	provider/scaling models and probes; scheduler APIs are deterministic doubles
Staging	`data_provider/data_manager.py`; file providers	transfer generation, publication version, DataFuture readiness; partial, atomic, stale	staging models and stage-out probes; file bytes are symbolic versions
Heartbeat/time	HTEX liveness and timer utilities	logical clock, deadline, heartbeat epoch; late ACK/rollback/timeout	clock models and probes; wall-clock and thread scheduling are abstract
Monitoring	`parsl/monitoring/`; database manager	event queue, batch, terminal observation; insert, retry, shutdown, malformed event	monitoring models and probes; database engine internals are finite records
`join_app`	join/future projection paths	outer task, inner futures, generation, failure aggregation; nested retry/-cancel	join models and probes; exception identity is abstract

Table 10: Cross-reference from implementation actions to abstract actions and invariants.

Concrete operation	Abstract action	Primary properties
Future dependency callback	AdmitReady(task)	DependencySafety, TerminalStability
Queue pop plus transport send	Dispatch(task, message)	OwnershipSafety, StaleIsolation
Worker launch and exit	StartAttempt / CompleteAttempt	RetryBound, ResultConsistency
Timeout and retry handler	AdvanceGeneration	RetryBound, TerminalStability
Multipart decode	Decode(message)	StaleIsolation, ResultConsistency
Result collector callback	DeliverResult(message)	ResultConsistency, TerminalStability
Provider submit/poll/cancel	Provision / Release(resource)	OwnershipSafety, RetryBound
Stage-in/stage-out publication	Publish(version)	ReadinessSafety, StaleIsolation
Heartbeat receive/ACK	ObserveLiveness(epoch)	TerminalStability, OwnershipSafety
Monitoring insert/update	RecordObservation(event)	TerminalStability, StaleIsolation
Memoized lookup	ResolveCache(key)	MemoizationSafety, TerminalStability
Nested join completion	Aggregate(inner futures)	ResultConsistency, RetryBound